

Indo para 2D

Malha · N · J · n · propgeo

Lucas S. Campos

Introdução ao MEC / BEM

“Indo para 2D”

“Indo para 2D - intro”

A partir desta aula o laboratório usa o repositório BEM_gmsh (Julia + Gmsh). O foco é **geometria de contorno**: malha, elementos, jacobiano, normal e integrais só em Γ . **Condições de contorno** (o que cada aresta “vale” em T ou q) ficam para o capítulo **Laplace 2D**.

Os trechos Julia abaixo são **pedaços do próprio pacote** (ou dos exemplos em data/examples/), para você reconhecer os mesmos nomes no código-fonte.

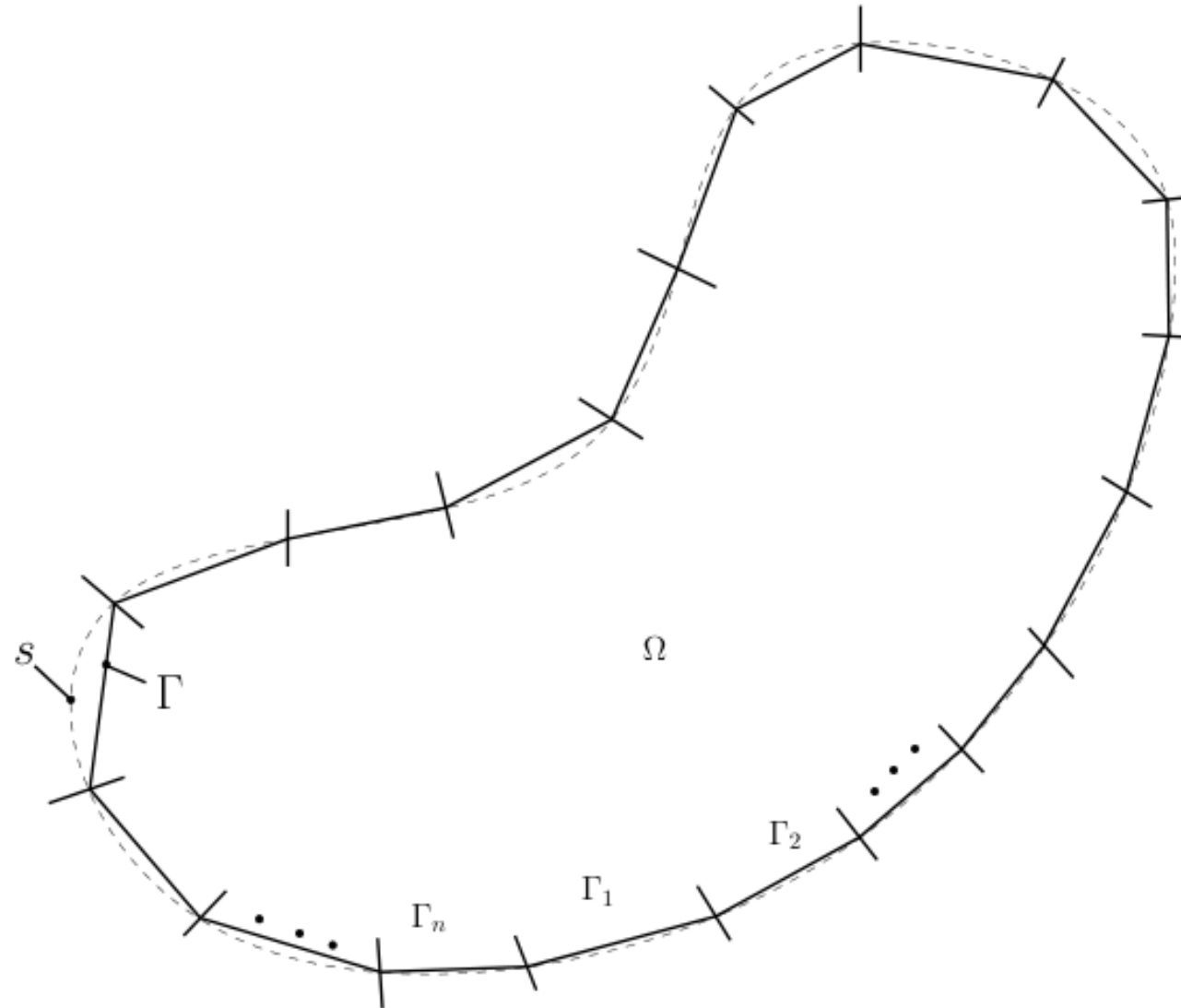
“Objetivos”

1. Ativar o ambiente BEM_gmsh e gerar uma malha de contorno com Gmsh.
2. Entender o elemento descontínuo de ordem p via `discontinuous_nodes_weights` + `shapefun` (Lagrange baricêntrico).
3. Calcular J , tangente e normal como em `format2d`.
4. Obter perímetro, área e centróide por **integração radial** (como `geometric_props`).
5. Reobter área (e análogos) pelo **teorema da divergência** e comparar.
6. Usar `geometric_props` / `geometric_props_2d_polygon` no fluxo do curso.

1. Por que 2D e o que muda em relação ao BEM 1D
2. Ambiente BEM_gmsh e Gmsh (geometria, sem CDC)
3. Elemento descontínuo + Lagrange baricêntrico (`Interpolation.jl` / `Input.jl`)
4. Jacobiano, normal (como em `format2d_lagrange`)
5. Propriedades geométricas — integração radial (`GeometricProperties.jl`)
6. O mesmo pelo teorema da divergência
7. Exercícios

No BEM 1D o “contorno” eram dois pontos. Em 2D:

- $\Omega \subset \mathbb{R}^2$ tem fronteira $\Gamma = \partial\Omega$ **unidimensional** (curva fechada, eventualmente com furos);
- a geometria de Γ é aproximada por **elementos de contorno** Γ_e ;
- campo e fluxo no contorno usam interpolação em $\xi \in [-1, 1]$ sobre cada Γ_e ;
- integrais de área em Ω que admitem redução a Γ (radial ou divergência) preparam as identidades do BEM.



Ideia-chave. Malhar só Γ não elimina a necessidade de boa geometria: N_k , J e $\mathbf{n}$ em cada elemento são os tijolos de `geometric_props` e, depois, de H e G .

Como em `scripts/intro.jl` e `data/examples/geo_unit_square.jl`:

```
using DrWatson
@quickactivate :BEM
include(datadir("Laplace", "Laplace_dad.jl"))
```

(Na primeira vez: `Pkg.activate` na pasta do clone + `Pkg.instantiate`.) O pacote `Gmsh.jl` já é dependência. A GUI do sistema (gmsh.info) ajuda a inspecionar `.geo` / `.msh`.

“Por que Gmsh no BEM (só geometria)”

1. Curvas, arcos, splines e furos com orientação controlável.
2. Malha de contorno (curvas) + superfície opcional (pontos internos / DIBEM depois).
3. Ordem geométrica alinhada ao grau do elemento (ordem / tipo).
4. O mesmo gerador alimenta Laplace, Poisson e elasticidade.

Nesta aula os grupos físicos **nomeiam** pedaços de curva e a superfície do domínio. A convenção " Γ " / " Ω " entra em **Laplace 2D**.

“Anatomia de um .geo (sem CDC)”

1. pontos e curvas;
2. **curve loop** + superfície plana;
3. grupos físicos de **geometria**;
4. Mesh 2.

```
lc = 0.15;

Point(1) = {0, 0, 0, lc};
Point(2) = {1, 0, 0, lc};
Point(3) = {1, 1, 0, lc};
Point(4) = {0, 1, 0, lc};

Line(1) = {1, 2};
Line(2) = {2, 3};
Line(3) = {3, 4};
Line(4) = {4, 1};

Curve Loop(1) = {1, 2, 3, 4}; // anti-horário = normal para fora
Plane Surface(1) = {1};

Transfinite Curve {1, 2, 3, 4} = 16;
Transfinite Surface{1};
Recombine Surface{1};

Physical Curve("contorno") = {1, 2, 3, 4};
Physical Surface("Domain") = {1};

Mesh 2;
```

```
gmsht -2 quadrado_geo.geo -o quadrado_geo.msh
```

No curso, preferimos geradores em `data/Laplace/Laplace_dad.jl` (ex.: `quadrado`, `placa_com_furo`). Fluxo geométrico mínimo — o mesmo de `data/examples/geo_unit_square.jl`:

```
using DrWatson
@quickactivate :BEM
include(datadir("Laplace", "Laplace_dad.jl"))

msh = quadrado(ndiv=10, show=false, nome="ex_geo_sq", ordem=2)
dad = format2d(msh, Laplace(1.0); tipo=2, pontointerno=false)
g = geometric_props(dad)
println("  perimeter = ", g.perimeter, "  (exact 4)")
println("  area      = ", g.area, "  (exact 1)")
println("  centroid   = ", g.centroid, "  (exact 0.5,0.5)")
plot_geo(dad)  # nós, elementos, normais – ainda sem solve
```

format2d lê a malha e monta BEMdata (elementos, nós de colocação, normais, pesos). **Nesta aula** use plot_geo e geometric_props, não solve / H_G*.

“Orientação (crítico)”

- Contorno **externo**: **anti-horário** $\rightarrow$ normal **para fora** de Ω .
- Contorno de **furo**: **horário** $\rightarrow$ normal ainda saindo de Ω .
- $A < 0$ nas fórmulas abaixo denuncia orientação invertida.

“O que format2d faz na geometria”

Trecho essencial de src/Core/Input.jl (format2d_lagrange):

```
# nós de colocação descontínuos = Gauss–Legendre (p+1 pontos)
qsi, wi = discontinuous_nodes_weights(p)    # = gausslegendre(p+1)

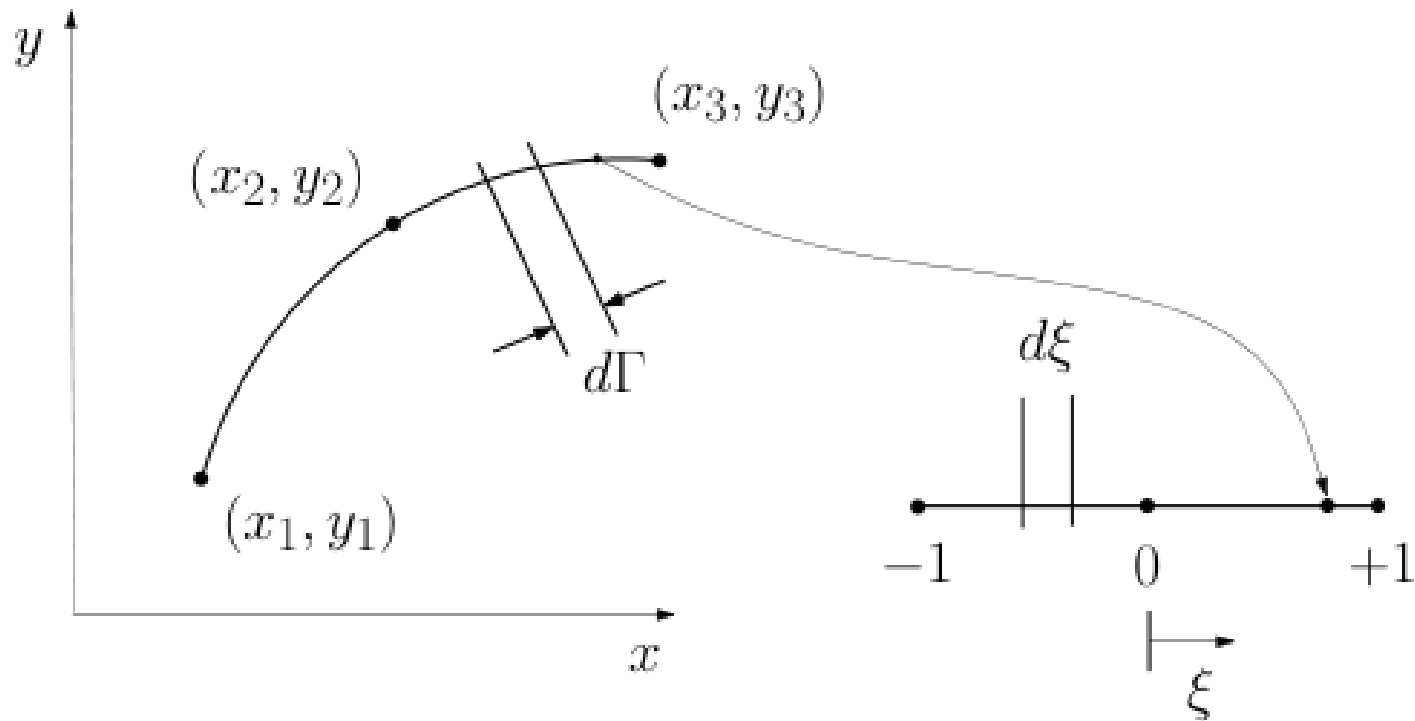
# geometria isoparamétrica nos nós Equispaced de grau p
Ngeo, dNgeo = shapefun(Equispaced(p), qsi)

# para cada aresta da malha Gmsh, com vértices X:
NOS[idx]  .= Ngeo * X           # posições dos nós de campo
dx        = dNgeo * X
J         = norm.(dx)
normal[idx] = tan2normal.(dx ./ J)
L         = abs(dot(J, wi))    # comprimento do elemento
```

Ou seja: **os mesmos** shapefun e pesos que você verá na montagem de H e G .

“Elemento de contorno descontínuo”

No BEM_gmsh o padrão é **elemento descontínuo**: graus de liberdade de campo **não** são compartilhados nos vértices entre elementos vizinhos (evita conflito de CDC em cantos — detalhe em Laplace 2D).



$$\Gamma_e$$

é a imagem de $\xi \in [-1, 1]$:

$$\mathbf{x}(\xi) = \sum_{k=1}^{p+1} N_k(\xi), \mathbf{x}_k^{(e)}.$$

- **Geometria** da aresta Gmsh: nós `Equispaced(p)` (vértices + meios, se ordem 2).
- **Campo** (colocação): nós `Legendre(p) = Gauss-Legendre` em $(-1, 1)$ via `discontinuous_nodes_weights(p)`.

```
# src/Core/Input.jl
function discontinuous_nodes_weights(p::Integer)
    p >= 1 || error("degree must be ≥ 1, got $p")
    return gausslegendre(p + 1)
end
```

`tipo=1,2,3` em `format2d` escolhe o grau p do campo (e alinha a ordem da malha se preciso).

“Lagrange baricêntrico (como no pacote)”

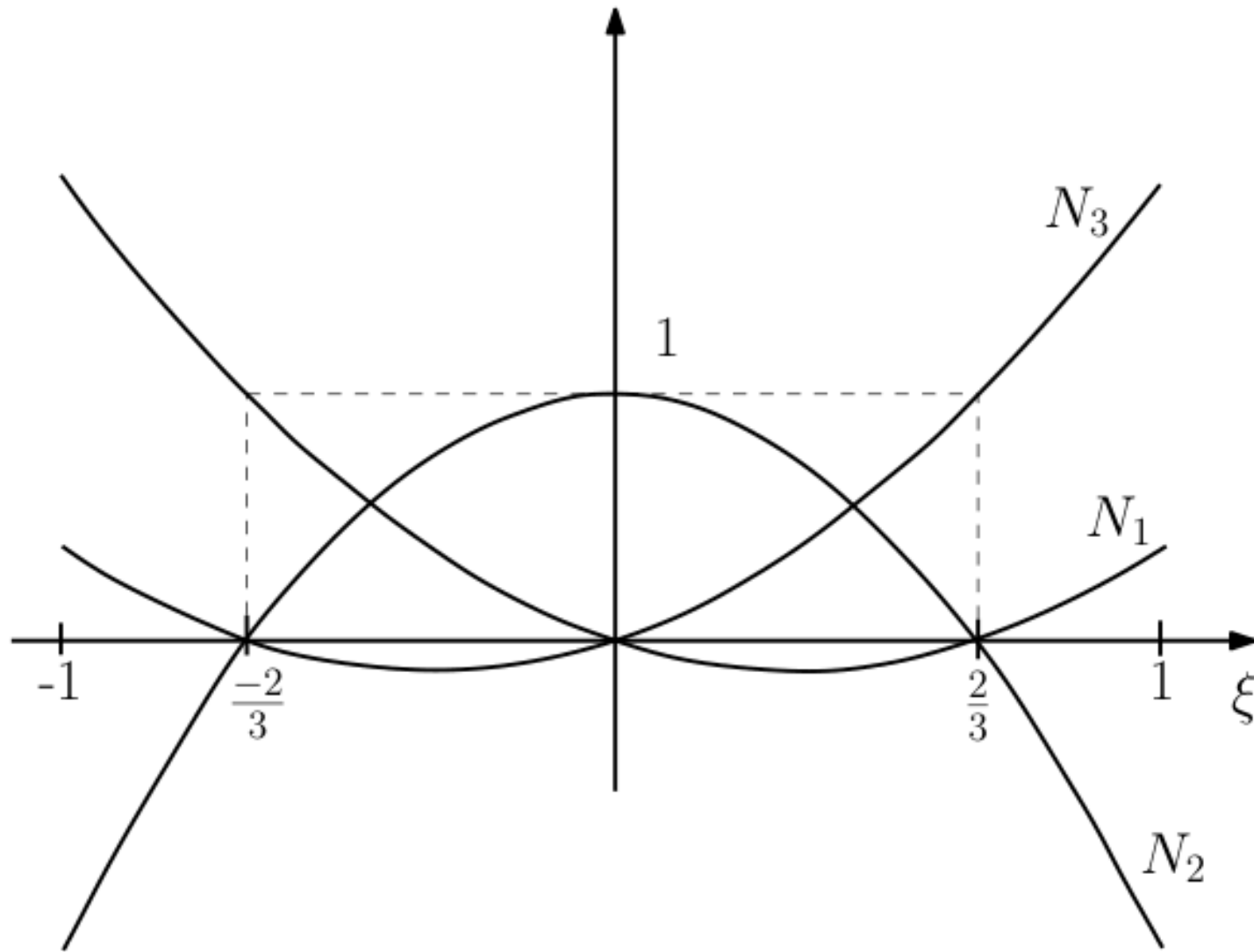
O núcleo está em `src/Core/Interpolation.jl`: pesos baricêntricos + matriz de interpolação (Berrut–Trefethen) e `shapefun` (= N e $dN/d\xi$ nos pontos pedidos).

Pesos (para nós x_0 do polinômio):

$$w_i = 1 / \prod_{j \neq i} (x_i - x_j).$$

Funções de forma no ponto ξ (fora dos nós):

$$N_i(\xi) = (w_i / (\xi - x_i)) / \sum_j (w_j / (\xi - x_j)).$$



No código:

```
# Interpolation.jl – interpolation_matrix (forma baricentrica)
# M[j,i] = w[i] / (xx - x0[i]), normalizado pela soma da linha
# se xx == no: linha de Kronecker

function shapefun(poly::AbstractPolynomial, x)
    L = interpolation_matrix(poly, x)    # N(x) nos pontos pedidos
    L, L * poly.Dmat                   # N e dN/dxi
end
```

“Dmat = differentiation_matrix(poly) (resumo)”

Com valores nodais $u_i = u(\xi_i)$, o interpolante $u(\xi) = \sum_i N_{i(\xi)} u_i$ tem

$$u'(\xi_k) = \sum_i N_{i'}(\xi_k) u_i \Rightarrow \mathbf{u}' = D\mathbf{u},$$

onde $D_{ki} = N_{i'}(\xi_k)$. No pacote, $D = \text{poly.Dmat}$, montada **uma vez** por `differentiation_matrix` (`Interpolation.jl`).

Com os pesos baricêntricos w_i e nós x_i :

$$D_{ki} = (w_i/w_k)/(x_k - x_i) \quad (k \neq i), \quad D_{kk} = -\sum_{i \neq k} D_{ki}$$

(a diagonal impõe $\sum_i N_{i'} = 0$: derivada de constante é zero).

`shapefun` devolve N e $dN/d\xi$ em pontos **quaisquer** ξ via

```
L = interpolation_matrix(poly, x)    # N
dN = L * poly.Dmat                 # N_i'(\xi) = \sum_k N_k(\xi) D_ki
```

Dmat **não** é o jacobiano da malha: age no interpolante 1D em ξ . O jacobiano geométrico vem depois, $dx/d\xi = \sum_i (dN_i/d\xi) \mathbf{X}_i$.

Uso didático (os mesmos tipos do `dad`):

```
using FastGaussQuadrature # já no projeto

p = 2
poly = Legendre(p)         # campo descontínuo (nós de Gauss)
xi, wN = nodes_weights(poly) # nós e pesos baricêntricos do interpolante
xi_g, wg = gausslegendre(6) # pontos de quadratura “de laboratório”
N, dN = shapefun(poly, xi_g) # size (6 × 3) se p=2

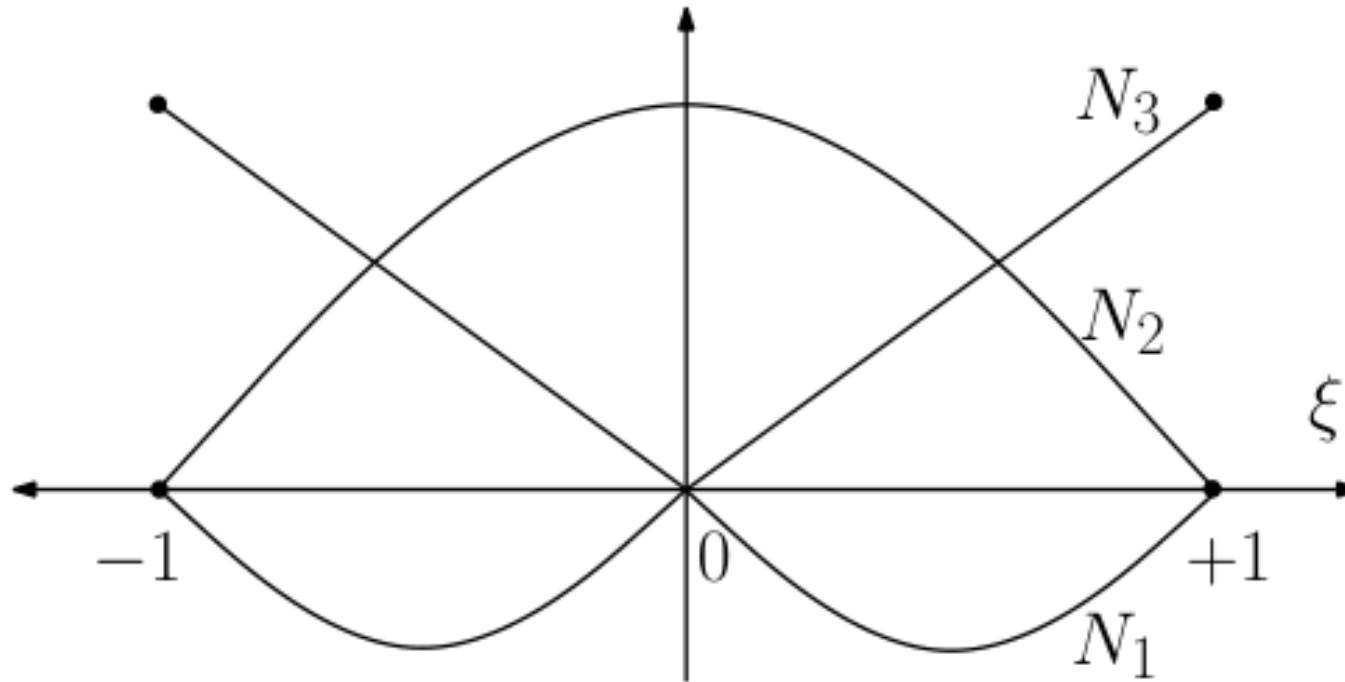
# geometria contínua da aresta (vértices Gmsh)
poly_geo = Equispaced(p)
Ngeo, dNgeo = shapefun(poly_geo, xi_g)
```

Partição da unidade (teste rápido):

```
poly = Legendre(3)
for ξ in range(-1, 1; length=21)
    N, dN = shapefun(poly, ξ)
    @assert abs(sum(N) - 1) < 1e-11
    @assert abs(sum(dN)) < 1e-9
end
```

Caso contínuo clássico $p = 2$, nós $\xi \in \{-1, 0, 1\}$ (Equispaced(2)):

$$N_1 = \xi(\xi - 1)/2, \quad N_2 = 1 - \xi^2, \quad N_3 = \xi(\xi + 1)/2.$$



No BEMdata. Depois de `format2d`, `dad.element_type` é o polinômio do campo (`Legendre(p)`), `dad.elem_weight` são os pesos de Gauss da colocação, e cada `dad.elements[e]` guarda índices dos nós, jacobianos nos nós de colocação e comprimento.

“Jacobiano, comprimento e normal”

Com nós geométricos $\mathbf{X}_k$ da aresta e N, N' em ξ :

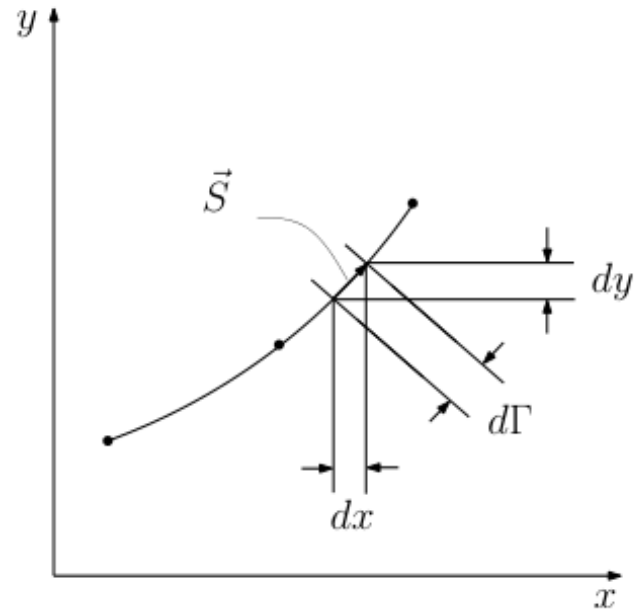
$$\mathbf{x}'(\xi) = \sum_k N_{k'}(\xi), \mathbf{X}_k, \quad J = |\mathbf{x}'|, \quad \mathbf{n} = (y', -x')/J$$

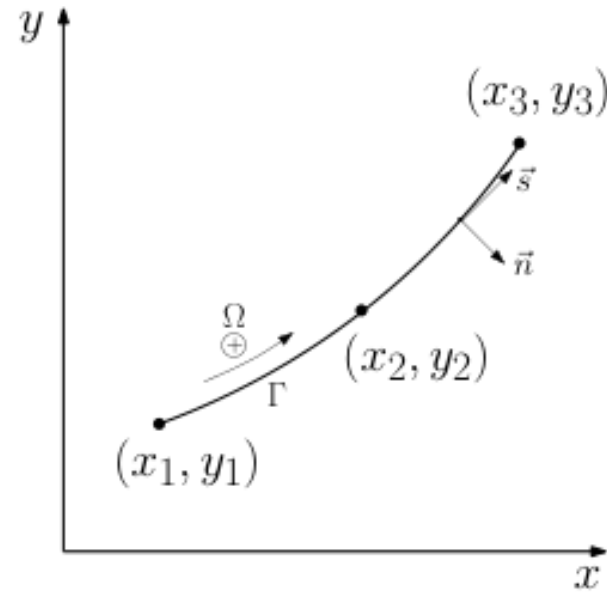
(normal “à esquerda” do sentido de percurso — contorno externo anti-horário $\Rightarrow$ exterior).

No pacote (format2d_lagrange + tan2normal):

```
dx = dNgeo * X
J = norm.(dx)
normal = tan2normal.(dx ./ J)    # (dx,dy) → (dy, -dx)/|·|  (unitário)
L = abs(dot(J, wi))              # ∫ J dξ ≈ Σ J_k w_k
```

Perímetro global: $P = \sum_e L_e$ (é o gp.perimeter).

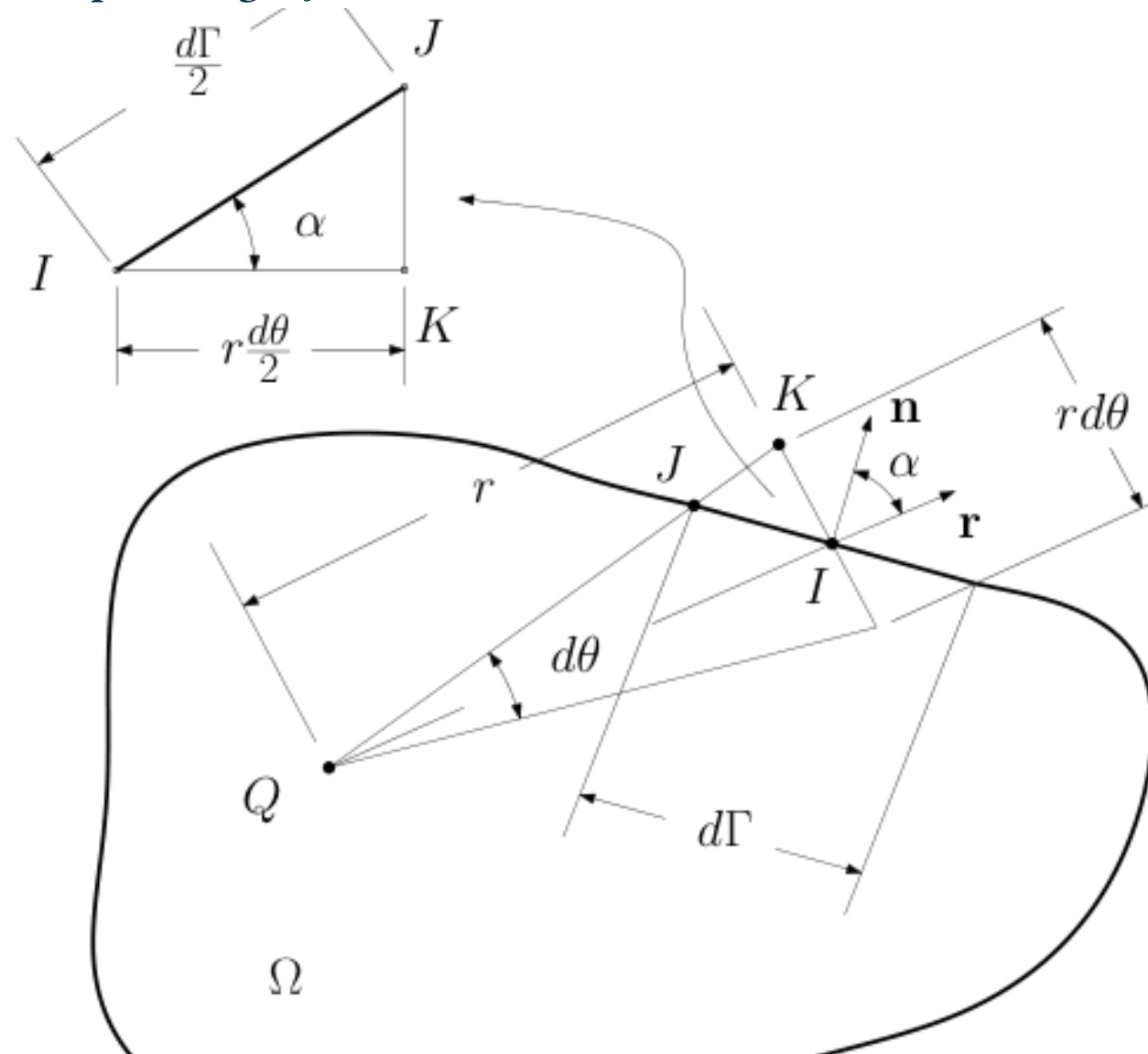


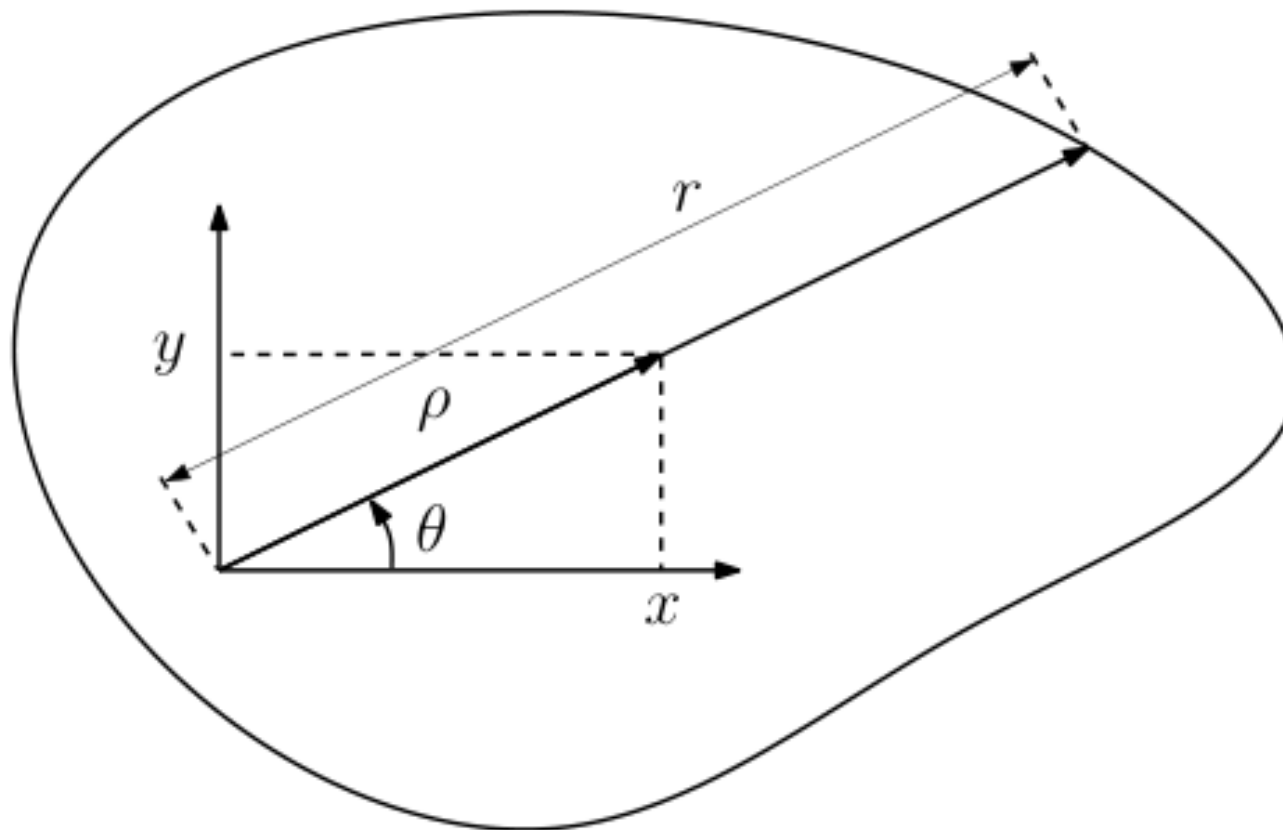


É a formulação de `geometric_props / geometric_props_2d` em `src/Core/GeometricProperties.jl`.

Fixe o pólo (no código, a **origem**). Para $\mathbf{x} \in \Gamma$, $\mathbf{r} = \mathbf{x}$, $r = |\mathbf{r}|$, $\hat{\mathbf{r}} = \mathbf{r}/r$.

$$dA = \rho, d\rho, d\theta, \quad d\theta = (\mathbf{n} \cdot \hat{\mathbf{r}})(d\Gamma)/r.$$





$$I = \int_{\Omega} f, dA = \int_{\Gamma} F(\mathbf{x}) \frac{\mathbf{n} \cdot \mathbf{r}}{r^2} d\Gamma,$$

$$F = \int_0^r f(\mathbf{x}_0 + \rho \hat{\mathbf{r}}), \rho, d\rho.$$

“Propriedades geométricas por integração radial”

No código, F para $f = 1$, $f = x$, $f = y$ é numérico na direção radial (`_calc_F_2d` com `npg_radial` pontos de Gauss em ρ):

```
# GeometricProperties.jl (essência do loop 2D)
# wJ = J * w no ponto de contorno
# nr = n ·  $\hat{r}$ 
# Fa, Fx, Fy =  $\int_{\partial\Omega} \{1, x, y\} \rho \, d\rho$ 
# A += Fa * nr / r * wJ
# xdA += Fx * nr / r * wJ
# ydA += Fy * nr / r * wJ
# centróide = (xdA, ydA) / A
```

API do aluno:

```
g = geometric_props(dad)           # default: quad. já no dad
g2 = geometric_props(dad; npg_boundary=12) # reintegra com shapefun + Gauss
g3 = geometric_props(dad; npg_radial=20)   # mais pontos na primitiva F

# polígono CCW sem Gmsh:
using StaticArrays
verts = [SA[0.0,0.0], SA[1.0,0.0], SA[1.0,1.0], SA[0.0,1.0]]
gp = geometric_props_2d_polygon(verts)
```

Leitura do fonte. Abra `geometric_props_2d`: o ramo `npg_boundary === nothing` reutiliza `elem.Jacobian` e `dad.elem_weight` (colocação); o outro chama `shapefun(dad.element_type,  $\xi$ )` — o mesmo `shapefun` da seção anterior.

Para $f = 1$ e pólo na origem recupera-se $A = \frac{1}{2} \int_{\Gamma} (\mathbf{n} \cdot \mathbf{x}) \, d\Gamma$ (quando a forma fechada vale). Momentos estáticos seguem de F com $f = x$ e $f = y$; centróide $|\mathbf{x}| = (S_x, S_y)/A$ como em `GeometricProps2D`.

“O mesmo pelo teorema da divergência”

$$\int_{\Omega} \nabla \cdot \mathbf{F}, dA = \int_{\Gamma} \mathbf{F} \cdot \mathbf{n}, d\Gamma.$$

F	$\nabla \cdot \mathbf{F}$	Resultado
$(x, 0)$ ou $(0, y)$	1	$A = \int_{\Gamma} x n_x d\Gamma$
$(x, y)/2$	1	$A = \frac{1}{2} \int_{\Gamma} \mathbf{x} \cdot \mathbf{n} d\Gamma$
$(x^2/2, 0)$	x	$\int_{\Omega} x dA = \int_{\Gamma} (x^2/2) n_x d\Gamma$
$(0, y^2/2)$	y	$\int_{\Omega} y dA = \int_{\Gamma} (y^2/2) n_y d\Gamma$
$(0, y^3/3)$	y^2	$I_x = \int_{\Omega} y^2 dA = \int_{\Gamma} (y^3/3) n_y d\Gamma$

Implementação **sobre o dad** (mesmos J , $\mathbf{n}$, pesos da colocação):

```

"""Área por divergência A = ∮ x n_x dΓ, usando a quad. já guardada no dad."""
function area_divergence_dad(dad)
    A = 0.0
    w_el = dad.elem_weight
    for elem in dad.elements
        for k in eachindex(elem.index)
            i = elem.index[k]
            x = dad.Nodes[i]
            n = dad.Normal[i]
            wJ = elem.Jacobian[k] * w_el[k]
            A += x[1] * n[1] * wJ
        end
    end
    return A
end

```

```
# comparar com a integração radial do pacote:  
g = geometric_props(dad)  
A_div = area_divergence_dad(dad)  
@show g.area A_div abs(g.area - A_div)
```

Radial × divergência. Para $f = 1$ coincidem (a menos de erro de quadratura). Radial generaliza f via F (é o que `geometric_props` faz para área/centróide). Divergência é imediata quando existe $\mathbf{F}$ polinomial simples (I_x , etc.).

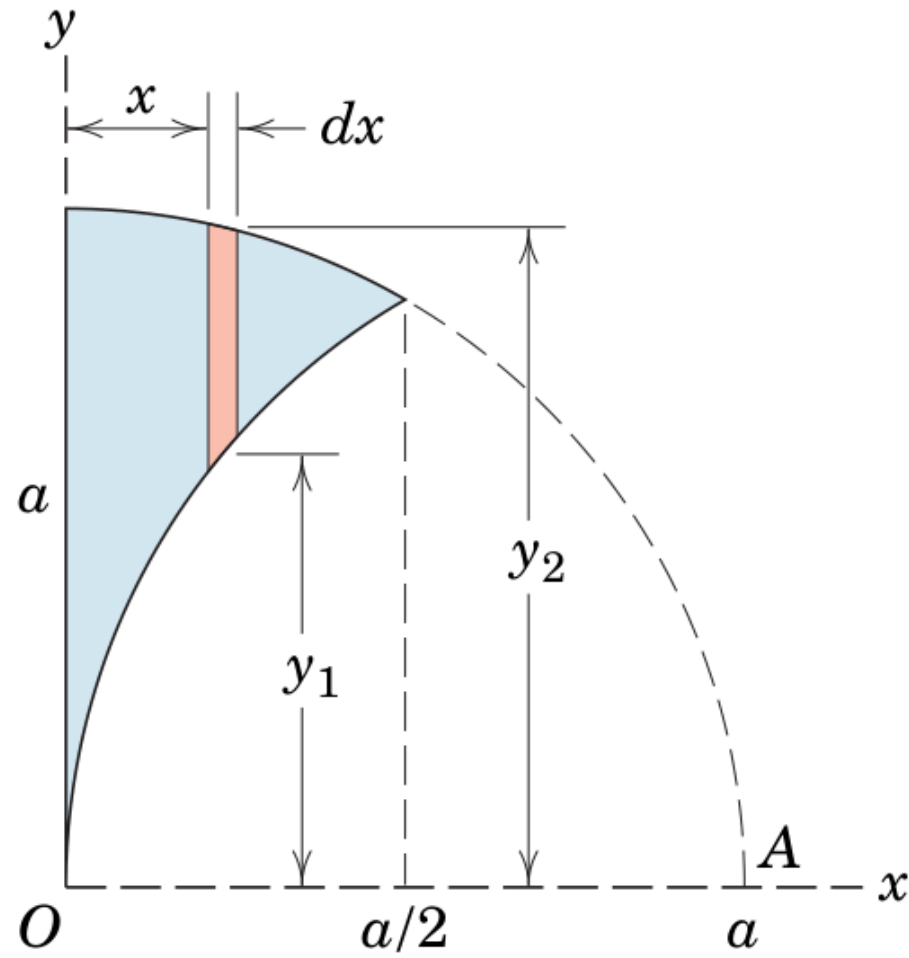
“Boas práticas de malha (geometria)”

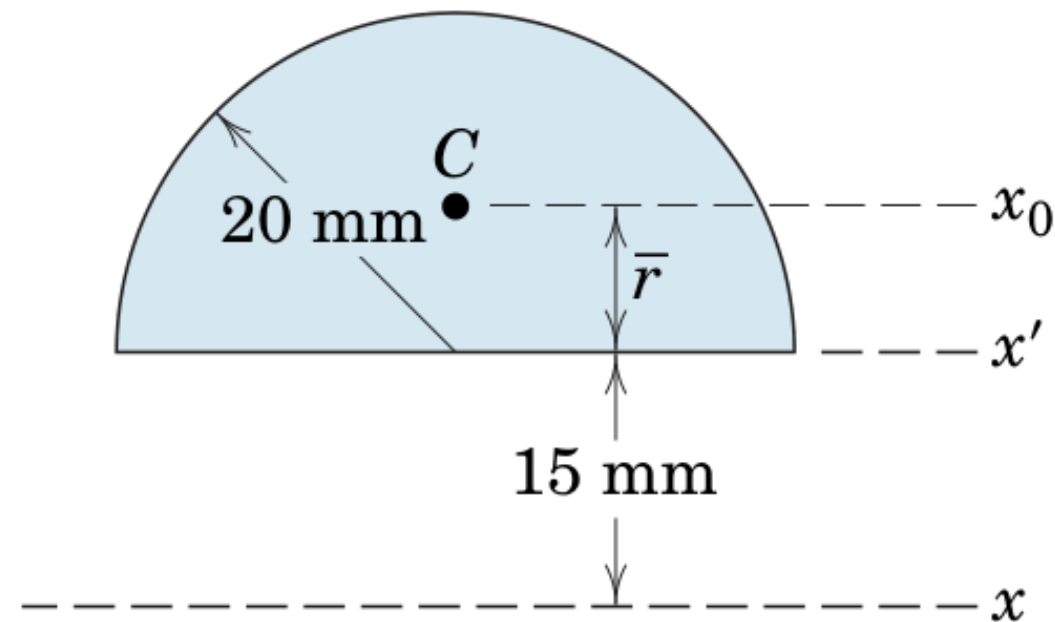
1. Contorno externo anti-horário; furos horários.
2. Elementos descontínuos no campo (padrão format2d) — cantos sem nó compartilhado de CDC.
3. ordem da malha Gmsh alinhada a tipo em format2d.
4. Refino perto de cantos e furos (placa_com_furo como modelo).
5. Nesta aula: `pontointerno=false` basta para prop. de Γ .

“Exercícios”

Entrega: scripts no ambiente BEM_gmsh, uso de `plot_geo` quando fizer sentido, números com erro vs analítico e 2–3 frases de interpretação. Fontes: `data/Laplace/Laplace_dad.jl`, `src/Core/{Input,Interpolation,GeometricProperties}.jl`, `data/examples/geo_unit_square.jl`.

1. **E1 — shapefun e partição da unidade.** Para $p \in \{1, 2, 3, 4\}$: (a) crie `poly = Legendre(p)` e, em 21 abscissas de $[-1, 1]$, verifique $\sum_i N_i = 1$ e $\sum_i N_{i'} = 0$ via `shapefun`; (b) plote as colunas de $N(\xi)$ (Plots) para $p = 2$ e $p = 3$; (c) com `Equispaced(2)` e $\xi \in \{-1, 0, 1\}$, confira N com $\xi(\xi - 1)/2$, $1 - \xi^2$, $\xi(\xi + 1)/2$; (d) compare `nodes(Legendre(p))` com `discontinuous_nodes_weights(p)[1]` — são o quê?
2. **E2 — Coroa circular: geometria, ordem e duas fórmulas de área.** Domínio $1 \leq r \leq 2$ (anel completo) ou o **quarto de coroa** do repo, se preferir adaptar um gerador em `Laplace_dad.jl` / API Gmsh. Valores analíticos do anel completo: $P = 2\pi(1 + 2) = 6\pi$, $A = \pi(2^2 - 1^2) = 3\pi$, centróide na origem.
 - (a) Gere malhas com `ndiv` in `{8,16,32}` e `tipo` in `{1,2}` (`format2d(..., tipo=...)`). Para cada par $(n_{\text{div}}, \text{tipo})$ obtenha `geometric_props(dad)` e os erros relativos de P e A .
 - (b) Com o **mesmo** `dad`, calcule a área por divergência (`area_divergence_dad` acima) e compare com `g.area`. Há diferença sistemática ao mudar `tipo`?
 - (c) Refaça `geometric_props(dad; npg_boundary=16)` e compare com o default (quadratura de colocação). Quando a reintegração muda o resultado de verdade?
 - (d) Plote `plot_geo(dad)` na malha mais grossa e na mais fina; comente se as normais no contorno interno (furo) apontam para fora de Ω .
 - (e) **Síntese:** tabela com colunas `ndiv`, `tipo`, `n_col` (`length(dad.Nodes)`), `e_P`, `e_A`, `e_Adiv`. Qual combinação atinge $e_A < 10^{-3}$ com menos nós de colocação?
3. **E3 — Círculo unitário e ordem em n .** Aproxime o disco unitário (malha Gmsh de um círculo ou polígono regular via `geometric_props_2d_polygon` nos vértices). Estude P e A vs 2π e π para n crescente e `tipo=1` vs `tipo=2`. Qual ordem aparente em n (ou em $h \sim 1/n$)?
4. **E4 — Momento I_x .**





Calcule $I_x = \int_{\Omega} y^2 dA$ nas duas figuras (divergência com $\mathbf{F} = (0, y^3/3)$ **ou** radial com $f = y^2$) e compare com $I_x = \frac{a^4}{96}(9\sqrt{3} - 2\pi)$ e $I_x = 2 \cdot 10^4 \pi - \frac{20^2 \pi}{2}(80/(3\pi))^2 + \left(\frac{20^2 \pi}{2}\right)(15 + 80/(3\pi))^2$. Reutilize malha Gmsh + laço no estilo de `geometric_props_2d` (nós, normais, wJ).

5. **E5 — Furo e orientação.** `placa_com_furo` (ou `.geo` próprio). (a) `geometric_props`: confira $A = A_{\text{ret}} - \pi R^2$ (e o perímetro $P_{\text{ret}} + 2\pi R$). (b) Inverta a orientação do furo no gerador e relate o sinal de A . (c) `plot_geo`: normais no contorno externo vs furo.

“O que fica para Laplace 2D”

- Convenção $(\theta; T)$, $(1; q)$ (e elasticidade $(t_x; u_x; t_y; u_y)$).
- `attach_analytical!`, `H_G_*`, `solve`, `rel_error`.
- CDC em cantos e o papel do elemento descontínuo no conflito de nós.

- `data/examples/geo_unit_square.jl`
- `src/Core/Input.jl` — `format2d`, `discontinuous_nodes_weights`
- `src/Core/Interpolation.jl` — pesos baricêntricos, `shapefun`
- `src/Core/GeometricProperties.jl` — `radial` + `geometric_props_2d_polygon`
- `data/Laplace/Laplace_dad.jl` — `quadrado`, `placa_com_furo`
- Manual Gmsh: documentação